

Esercizio 1 – pipeline di comandi

Scrivere una pipeline di comandi che stampi la lista di tutti gli utenti che hanno almeno un processo con almeno 2 thread (LWP). Ogni utente deve comparire al massimo una volta nell'output.

Comandi utili: ps, awk, sort.

Suggerimento: il comando `ps -eo user,nlwp` stampa, per ogni processo, l'utente (user) e il numero di thread (NLWP):

```
ps -eo user,nlwp
```

```
USER      NLWP
root       2
albert     1
root       1
albert     2
alice      1
```

Esercizio 2

Dato l'output di: `ls -l`

Esempio:

```
-rw-r--r-- 1 student student 9200 Nov 12 10:10 relazione.txt
-rw-r----- 1 student student 4500 Nov 12 10:11 appunti.txt
-rw-rw-r-- 1 student student 12000 Nov 12 10:12 dati.txt
-rwxr-xr-x 1 student student 2100 Nov 12 10:13 script.sh
drwxr-xr-x 2 student student 4096 Nov 12 10:14 documenti
-r--r----- 1 student student 1800 Nov 12 10:15 note.txt
-rw-r--r-- 1 alice staff 512 Nov 12 10:16 elenco.txt
lrwxrwxrwx 1 student student 15 Nov 12 10:17 link -> relazione.txt
```

si vogliono visualizzare i nomi dei file che soddisfano tutte le seguenti condizioni:

- sono file regolari (non directory, non link, ecc.);
- hanno estensione .txt;
- sono leggibili sia dal proprietario (user) sia dal gruppo (group).

Scrivere una pipeline di comandi per ottenere l'elenco dei soli nomi dei file che soddisfano queste condizioni.

Comandi utili: `ls -l`, `grep`, `awk`.

Esercizio 3

Si vuole realizzare un meccanismo che permette a **non più di 3 thread** alla volta di entrare in una sezione critica. Sono definite le seguenti variabili globali:

```
int dentro = 0; // thread attualmente in sezione critica

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

Ogni thread lancia la funzione worker definita come segue:

```
void sezione_critica(void);
```

```

void *worker(void *arg) {
    while (1) {
        inizio();
        sezione_critica();
        fine();
    }
}

```

Completare le funzioni **inizio()** e **fine()** inserendo le chiamate corrette a `pthread_mutex_lock`, `pthread_mutex_unlock`, `pthread_cond_wait`, `pthread_cond_signal` (o `pthread_cond_broadcast`) in modo che un massimo di 3 thread contemporaneamente possono entrare in sezione critica mentre gli altri restano in attesa sulla variabile di condizione.

Suggerimento sulla struttura di inizio e fine:

```

void inizio(void) {
    /* A */
    while (/* B: condizione di attesa */) {
        /* C */
    }
    /* D: aggiornare thread entrati */
    /* E */
}

void fine(void) {
    /* F */
    /* G: aggiornare thread usciti */
    /* H: svegliare eventuali thread in attesa */
    /* I */
}

```

Esercizio 4

Si consideri il seguente programma C che crea due processi figli. Ogni figlio scrive periodicamente un carattere su una pipe verso il padre: il figlio 1 scrive 'A' sulla prima pipe e il figlio 2 scrive 'B' sulla seconda pipe. Il processo padre deve usare `select()` per leggere dai due descrittori di lettura delle pipe. Il padre deve inoltre contare quanti caratteri riceve da ciascuna pipe; terminare dopo aver ricevuto in totale 10 caratteri (sommando entrambe le pipe); stampare a fine esecuzione quanti caratteri sono arrivati da ciascuna pipe.

3.a Completare le parti indicate:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/select.h>

int main(void) {
    int p1[2], p2[2];
    char buf;
    int c1 = 0, c2 = 0;

    pipe(p1); pipe(p2);

```

```

if (fork() == 0) {
    //TODO: chiude pipe p1 in lettura
    while (1) write(p1[1], "A", 1);
}
if (fork() == 0) {
    //TODO: chiude pipe p2 in lettura
    while (1) write(p2[1], "B", 1);
}
close(p1[1]); close(p2[1]);

fd_set set;
int maxfd = /* TODO: assegnare*/;

while (c1 + c2 < 10) {
    FD_ZERO(&set);
    /* TODO: aggiungere pipe al set di fd */
    select(/*TODO: mettere argomenti del select */);
    /* TODO: controllare pipe pronte e incrementare
    i contatori dei caratteri ricevuti */
}
printf("pipe1 = %d, pipe2 = %d\n", c1, c2);
close(p1[0]); close(p2[0]);
return 0;
}

```

3.b Aggiungere la gestione delle pipe rotte per i figli mediante handler del SIGPIPE

Esercizio 5 Dato il frammento di programma che segue:

- Quanti processi e quanti thread genera il frammento di programma seguente?
- Il programma termina correttamente? Segnalare e correggere eventuali anomalie
- Cosa stampa il programma?

```

#define NUM_THREADS 2
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
int v1 = 1, v2 = 5;

void *f1(void* param){
    pthread_mutex_lock(&m);
    while (v1 != v2)
        pthread_cond_wait(&c, &m);
}

```

```

    v1 = 1 + v2;
    pthread_mutex_unlock(&m);
    return 0;
}

void *f2(void* param){
    pthread_mutex_lock(&m);
    v2 = v1+1; v1++;
    pthread_mutex_unlock(&m);
    return 0;
}

int main(){
    pthread_t threads[NUM_THREADS];
    int pid;
    pid = fork();
    if (pid != 0) {
        pthread_create(&threads[0], NULL, f1, NULL);
        pthread_create(&threads[1], NULL, f2, NULL);
        pthread_join(threads[0], NULL);
        pthread_join(threads[1], NULL);
    }
    printf("%d %d\n", v1, v2);

    return 0;
}

```